

Contents

SECTION 16: AWS ADMIN CREDENTIALS (v3.2.0)	1
16.1 AWS Admin Integration	1
	2

SECTION 16: AWS ADMIN CREDENTIALS (v3.2.0)

16.1 AWS Admin Integration

Admin-level AWS credential management for deployment operations.

```
// packages/core/src/services/aws-admin-credentials.ts
```

```
import { STSClient, AssumeRoleCommand, GetCallerIdentityCommand } from '@aws-sdk/client-sts';
import { IAMClient, CreateAccessKeyCommand, DeleteAccessKeyCommand, ListAccessKeysCommand } from '@aws-sdk/client-iam';
```

```
interface AssumedCredentials {
  accessKeyId: string;
  secretAccessKey: string;
  sessionToken: string;
  expiration: Date;
}
```

```
export class AwsAdminCredentials {
  private sts: STSClient;
  private iam: IAMClient;

  constructor() {
    this.sts = new STSClient({});
    this.iam = new IAMClient({});
  }
}
```

```
async assumeDeploymentRole(
  roleArn: string,
  sessionName: string,
  durationSeconds: number = 3600
): Promise<AssumedCredentials> {
  const response = await this.sts.send(new AssumeRoleCommand({
    RoleArn: roleArn,
    RoleSessionName: sessionName,
    DurationSeconds: durationSeconds
  }));
}
```

```

    if (!response.Credentials) {
        throw new Error('Failed to assume role');
    }

    return {
        accessKeyId: response.Credentials.AccessKeyId!,
        secretAccessKey: response.Credentials.SecretAccessKey!,
        sessionToken: response.Credentials.SessionToken!,
        expiration: response.Credentials.Expiration!
    };
}

async validateCredentials(): Promise<{ accountId: string; arn: string }> {
    const response = await this.sts.send(new GetCallerIdentityCommand({}));
    return {
        accountId: response.Account!,
        arn: response.Arn!
    };
}

async rotateAccessKeys(userName: string): Promise<{ accessKeyId: string; secretAccessKey: string }> {
    // List existing keys
    const listResponse = await this.iam.send(new ListAccessKeysCommand({ UserName: userName }));

    // Create new key
    const createResponse = await this.iam.send(new CreateAccessKeyCommand({ UserName: userName }));

    // Delete old keys (keep only the new one)
    for (const key of listResponse.AccessKeyMetadata || []) {
        if (key.AccessKeyId !== createResponse.AccessKey!.AccessKeyId) {
            await this.iam.send(new DeleteAccessKeyCommand({
                UserName: userName,
                AccessKeyId: key.AccessKeyId
            }));
        }
    }

    return {
        accessKeyId: createResponse.AccessKey!.AccessKeyId!,
        secretAccessKey: createResponse.AccessKey!.SecretAccessKey!
    };
}
}

```